

Resource Configuration in NOAA Operational Workflows: A Technical Comparison of CROW and Global-Workflow Approaches

National Oceanic and Atmospheric Administration
Environmental Modeling Center

January 2026

Abstract

This technical report provides a detailed comparison of resource configuration methodologies between the Community Research Operations Workflow (CROW) system and the current production global-workflow. We examine the architectural differences in how computational resources—CPU ranks, threads, memory, walltime, and MPI execution—are specified, validated, and deployed across heterogeneous HPC platforms. Through side-by-side code analysis, we demonstrate the trade-offs between CROW’s declarative YAML-based approach and global-workflow’s imperative shell-based configuration. This comparison serves as a reference for developers maintaining the current system and architects designing next-generation workflow infrastructure.

1 Introduction

The Global Forecast System (GFS) requires precise allocation of computational resources across hundreds of interdependent jobs. A single forecast cycle involves atmospheric dynamics, physics parameterizations, data assimilation, post-processing, and product generation—each with distinct resource requirements that vary by model resolution (C96 through C768) and HPC platform (WCOSS2, HERA, Orion, Hercules, Gaea).

This report examines two fundamentally different approaches to specifying these resources:

1. **CROW (2016-2020)**: Declarative YAML with custom type extensions
2. **Global-Workflow (2020-present)**: Imperative shell scripts with environment variables

2 Resource Specification Paradigms

2.1 CROW: Declarative Resource Tables

CROW centralized resource definitions in `workflow/defaults/default_resources.yaml` using resolution-indexed lookup tables.

2.1.1 Resource Table Structure

Listing 1: CROW GFS Resource Table

```
gfs_resource_table: !Select
  select: !calc doc.fv3_gfs_settings.CASE
  otherwise: !error Unknown GFS case
```

```

cases:
  C96: &gfs_resource_C96
    - [24, 12, '00:15:00', 1, '1024M']
  C192: &gfs_resource_C192
    - [48, 12, '00:25:00', 1, '1024M']
  C384: &gfs_resource_C384
    - [120, 12, '00:45:00', 2, '2048M']
  C768: &gfs_resource_C768
    - [240, 12, '01:30:00', 4, '4096M']

```

The tuple format encodes: [ranks, ppn, walltime, threads, memory].

Key features:

- **!Select** — Pattern matching on configuration value
- **!calc** — Runtime expression evaluation
- **!error** — Explicit failure for undefined cases
- **YAML anchors (&)** — Reusable definitions

2.1.2 JobRequest Construction

Resources were consumed via **!JobRequest** objects:

Listing 2: CROW JobRequest Pattern

```

gfs_forecast: !JobRequest
- batch_memory: !calc >-
  doc.gfs_resource_table[0][4]
- mpi_ranks: !calc >-
  doc.gfs_resource_table[0][0]
- OMP_NUM_THREADS: !calc >-
  doc.gfs_resource_table[0][3]
- walltime: !calc >-
  tools.parse_duration(
    doc.gfs_resource_table[0][2])
- max_ppn: !calc >-
  doc.gfs_resource_table[0][1]

```

2.1.3 Downstream Resource Tables

CROW maintained separate tables for post-processing steps with different resource profiles:

Listing 3: CROW Downstream Resources

```

downstream_resource_table: !Select
select: !calc doc.fv3_gfs_settings.CASE
cases:
  C96:
    awips: [24, 1, '00:05:00', 1, '2G']
    gempak: [24, 1, '00:05:00', 1, '2G']
    bufrsnd: [80, 8, '00:20:00', 1, '3G']
  C768:
    awips: [24, 1, '00:10:00', 1, '4G']
    gempak: [24, 1, '00:10:00', 1, '4G']
    bufrsnd: [80, 8, '00:45:00', 1, '6G']

```

2.2 Global-Workflow: Imperative Configuration

The current system uses shell scripts with conditional logic in `parm/config/config.resources`.

2.2.1 Resource Selection Logic

Listing 4: Global-Workflow Resource Selection

```
case "${step}" in
"fcst")
  case "${CASE}" in
    "C48")
      export ntasks=24
      export ntasks_quilt=6
      export nth_fv3=1
      export nth_quilt=1
      ;;
    "C96")
      export ntasks=48
      export ntasks_quilt=12
      export nth_fv3=2
      export nth_quilt=1
      ;;
    "C384")
      export ntasks=150
      export ntasks_quilt=36
      export nth_fv3=4
      export nth_quilt=1
      ;;
    "C768")
      export ntasks=360
      export ntasks_quilt=72
      export nth_fv3=4
      export nth_quilt=1
      ;;
  esac
  ;;
"anal")
  # Analysis step resources
  export ntasks=480
  export nth=8
  ;;
esac
```

2.2.2 Platform-Specific Launchers

MPI execution is configured in platform environment files:

Listing 5: HERA Platform Configuration (env/HERA.env)

```
# HERA - Slurm/srun based
ulimit_s="unlimited"
launcher="srun -l --export=ALL"
mpmd_opt="--multi-prog"

# Construct APRUN for each component
export APRUN_UFS="${launcher} -n ${ntasks}"
export APRUN_GAUSSIAN="${launcher} -n ${ntasks} --cpus-per-task=${nth}"
export APRUN_CHGRES="${launcher} -n 6"
```

Listing 6: WCOSS2 Platform Configuration (env/WCOSS2.env)

```
# WCOSS2 - PBS/mpirun based
ulimit_s="unlimited"
launcher="mpirun -l"
mpmd_opt="--cpu-bind verbose,core cfp"

# Construct APRUN for each component
export APRUN_UFS="${launcher} -np ${ntasks}"
```

```
export APRUN_GAUSSIAN="${launcher} -np ${ntasks}"
export APRUN_CHGRES="${launcher} -np 6"
```

3 MPI Execution Patterns

3.1 CROW: Abstract Parallelism Layer

CROW generated scheduler-agnostic job specifications through platform configuration:

Listing 7: CROW Platform Abstraction

```
hera: !Platform
  name: hera
  scheduler_settings:
    scheduler_name: slurm

  partitions:
    default: &default_partition
    specification: !icalc |
      tools.get_parallelism('HyperThread')
    scheduler_settings:
      partition: hera
      qos: batch
```

The CROW engine would translate `!JobRequest` objects into appropriate Slurm or PBS directives at generation time.

3.2 Global-Workflow: MPMD Wrapper

The current system uses `ush/run_mpmd.sh` for MPMD execution:

Listing 8: MPMD Execution Wrapper

```
#!/bin/bash
# run_mpmd.sh - Execute MPMD job

create_mpmd_cmdfile() {
  local cmdfile=$1
  shift
  local nprocs
  while (( $# > 0 )); do
    nprocs=$1; shift
    cmd=$1; shift
    for (( i=0; i<nprocs; i++ )); do
      echo "$cmd" >> "${cmdfile}"
    done
  done
}

run_mpmd() {
  local cmdfile=$1

  case "${launcher_type:-srun}" in
    "srun")
      ${APRUN_CMD} --multi-prog "${cmdfile}"
      ;;
    "mpiexec")
      ${APRUN_CMD} cfp "${cmdfile}"
      ;;
  esac
}
```

3.3 CFP (Coupled Framework Parallelism)

Both systems support CFP for running multiple executables in a single MPI job:

Listing 9: CFP Command File Format

```
# cmdfile format: one command per MPI rank
/path/to/executable1 arg1 arg2
/path/to/executable1 arg1 arg2
/path/to/executable1 arg1 arg2
/path/to/executable2 arg1
/path/to/executable2 arg1
```

4 Comparative Analysis

4.1 Code Organization

Table 1 compares where resource information resides in each system.

Table 1: File Organization Comparison

CROW	Global-Workflow
workflow/defaults/	parm/config/
default_resources.yaml	config.resources
workflow/platforms/	env/HERA.env
hera.yaml	
workflow/platforms/	env/WCOSS2.env
wcoss_dell_p3.yaml	
Generated at run-time	ecf/scripts/*.ecf (PBS/Slurm headers)

4.2 Adding a New Resolution

4.2.1 CROW Approach

Add one entry to each resource table:

Listing 10: Adding C1152 to CROW

```
gfs_resource_table: !Select
cases:
  # ... existing cases ...
  C1152: &gfs_resource_C1152
    - [480, 12, '02:30:00', 4, '8192M']
```

All jobs using `gfs_resource_table` automatically inherit the new resolution.

4.2.2 Global-Workflow Approach

Add case branches to each step in `config.resources`:

Listing 11: Adding C1152 to Global-Workflow

```
case "${step}" in
```

```

"fcst")
  case "${CASE}" in
    # ... existing cases ...
    "C1152")
      export ntasks=480
      export ntasks_quilt=96
      export nth_fv3=4
      export nth_quilt=1
      ;;
    esac
  ;;
"anal")
  case "${CASE}" in
    # ... must add C1152 here too ...
    esac
  ;;
# ... repeat for each step ...
esac

```

4.3 Adding a New HPC Platform

4.3.1 CROW Approach

Create a new platform YAML file:

Listing 12: New Platform in CROW

```

# workflow/platforms/new_hpc.yaml
new_hpc: !Platform
  name: new_hpc
  scheduler_settings:
    scheduler_name: slurm # or pbs

  partitions:
    default:
      specification: !icalc |
        tools.get_parallelism('NoHyperThread')
      scheduler_settings:
        partition: compute
        qos: normal

```

4.3.2 Global-Workflow Approach

Create a new environment file:

Listing 13: New Platform in Global-Workflow

```

# env/NEW_HPC.env
ulimit_s="unlimited"
launcher="srun -l --export=ALL"
mpmd_opt="--multi-prog"

# Set all APRUN variables
export APRUN_UFS="${launcher} -n ${ntasks}"
export APRUN_GAUSSIAN="${launcher} -n ${ntasks}"
# ... 15+ more APRUN definitions ...

```

Additionally, update ecFlow job scripts with platform-specific PBS/Slurm headers.

4.4 Validation Capabilities

Table 2: Validation Feature Comparison

Feature	CROW	GW
Schema validation	Yes (YAML)	No
Type checking	Yes (custom)	No
Undefined case detection	!error	Runtime
Static analysis	Possible	Limited
IDE support	Limited	Good
Syntax highlighting	YAML	Bash

5 Resource Tuple Semantics

Both systems encode the same fundamental resource parameters, but with different representations.

5.1 CROW Tuple Format

[ranks, ppn, walltime, threads, memory]

Example: [240, 12, '01:30:00', 4, '4096M']

- 240 MPI ranks total
- 12 processes per node (ppn)
- 1 hour 30 minute walltime limit
- 4 OpenMP threads per rank
- 4 GB memory per node

5.2 Global-Workflow Variables

Listing 14: Equivalent Variables

```
export ntasks=240      # MPI ranks
export ppn=12          # Processes per node
export walltime="01:30:00" # Walltime
export nth_fv3=4       # OMP threads
export memory="4GB"    # Memory
```

6 Execution Flow Comparison

6.1 CROW Execution Path

1. Parse YAML configuration files
2. Evaluate **!calc** expressions
3. Resolve **!Select** based on CASE

4. Construct `!JobRequest` objects
5. Generate scheduler scripts (Slurm/PBS)
6. Submit to workflow engine (ecFlow/Rocoto)

6.2 Global-Workflow Execution Path

1. Source platform environment (`env/*.env`)
2. Source `config.resources`
3. Evaluate shell conditionals
4. Export environment variables
5. ecFlow job script reads variables
6. Submit with embedded PBS/Slurm directives

7 Memory and Thread Configuration

7.1 CROW Memory Specification

CROW supported both per-node and per-rank memory:

Listing 15: CROW Memory Options

```
gfs_forecast: !JobRequest
- batch_memory: '4096M'    # Per node
# OR
- memory_per_rank: '1024M' # Per rank
```

7.2 Global-Workflow Memory

Memory is platform-dependent and often implicit:

Listing 16: Global-Workflow Memory Handling

```
# In config.resources
export memory="4GB"

# In ecf script
#PBS -l select=1:mem=${memory}
# OR
#SBATCH --mem=${memory}
```

8 Error Handling

8.1 CROW: Fail-Fast

Listing 17: CROW Error Handling

```
gfs_resource_table: !Select
select: !calc doc.settings.CASE
otherwise: !error |
```



```
Unknown resolution: expected C96-C768
cases:
  C96: [...]
```

Invalid configurations fail at YAML parse time, before job submission.

8.2 Global-Workflow: Runtime Defaults

Listing 18: Global-Workflow Default Handling

```
case "${CASE}" in
  "C96") export ntasks=48 ;;
  "C384") export ntasks=150 ;;
  *)
    echo "WARNING: Unknown CASE ${CASE}"
    export ntasks=24 # Default fallback
    ;;
esac
```

Unknown configurations may proceed with potentially incorrect defaults.

9 Practical Implications

9.1 Maintenance Burden

Table 3: Lines of Code for Resource Configuration

Component	CROW	GW
Core resource definitions	~200	~800
Platform configurations	~150/platform	~200/platform
Job request constructors	~100	N/A
Total (5 platforms)	~1,050	~1,800

9.2 Debugging Experience

CROW: Requires understanding of YAML processor, custom tags, and expression evaluation. Stack traces reference YAML line numbers.

Global-Workflow: Standard bash debugging with `set -x`, `echo` statements, and familiar tools. Errors reference shell line numbers.

9.3 Onboarding Time

CROW: 2-4 weeks to become proficient with custom DSL.

Global-Workflow: 1-2 weeks for developers familiar with bash.

10 Summary Comparison

11 Conclusion

CROW and global-workflow represent two valid approaches to the complex problem of HPC resource configuration. CROW’s declarative approach minimized duplication and enabled static validation,

Table 4: Comprehensive Feature Comparison

Dimension	CROW	Global-Workflow
Configuration paradigm	Declarative (YAML DSL)	Imperative (Bash)
Resource definition	Centralized tables	Distributed case statements
Resolution scaling	Single entry per table	Entry per step per resolution
Platform abstraction	Complete (runtime generation)	Partial (per-platform env files)
Scheduler support	Slurm, PBS, LSF (pluggable)	Slurm, PBS (per-platform)
Type system	Rich (<code>!JobRequest</code> , <code>!icalc</code>)	None (strings)
Validation	Parse-time schema checking	Runtime only
Error handling	Fail-fast (<code>!error</code>)	Fallback defaults
IDE support	Limited (custom YAML tags)	Excellent (bash)
Debugging	Multi-layer indirection	Direct script inspection
Learning curve	Steep (2-4 weeks)	Moderate (1-2 weeks)
Code duplication	Minimal (DRY)	Moderate (WET)
GPU support	Not implemented	Not implemented
Cloud readiness	Limited	Limited

but required specialized knowledge. Global-workflow’s imperative approach traded some elegance for transparency and debuggability.

Neither system currently addresses emerging requirements for GPU orchestration, cloud deployment, or machine learning workflow integration. Future infrastructure should consider hybrid approaches that combine:

- Declarative resource specification (like CROW) for consistency
- Standard tooling (unlike CROW’s custom tags) for maintainability
- Schema validation for early error detection
- Explicit compilation to shell for debuggability

Acknowledgments

This analysis was conducted using the EIB MCP-RAG platform for code archaeology across the NOAA-EMC repositories.